01

MGU BCA - 3RD SEM

Python Programming

Module 1 - Introduction, Strings, Control Flow

By Aben Sojan

Exam - 16 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).
Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.
Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.
Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.
Module 1 in the model paper: indentation, = vs ==, slicing (Part A); leap year program (Part B); loop control statements (Part C).

TOPIC 1

History and Application Areas of Python

1

Python was created by Guido van Rossum and released in 1991.

It is named after the comedy show Monty Python, not the snake.

Python is a high-level, interpreted, general-purpose programming language.

It is free and open source with a huge collection of ready-made libraries.

Used in web development, data science, AI and machine learning, automation and game development.

QUESTION BANK

1 **Explain the key features of Python. (MG2DSCCSC100, Nov 2025)**

REPEATED

Python is simple and easy to learn: its syntax reads almost like plain English, so beginners write working programs quickly.

Python is interpreted: code runs line by line without a separate compilation step, which makes testing and debugging fast.

Python is free and open source: anyone can download, use and modify it at no cost.

Python is portable: the same program runs on Windows, Linux and Mac with little or no change.

Python has a huge standard library: ready-made modules exist for files, maths, dates, web work and more.

Python is dynamically typed: you never declare variable types, Python decides the type at run time.

Example:

```
x = 10
```

```
print(type(x))   # dynamically typed: no declaration needed
```

A large community and libraries like Numpy and Django mean almost every problem already has a Python solution.

These features are why Python is the first choice for beginners and for fields like data science and AI.

2 Explain the application areas of Python.

IMPORTANT

Python is a general-purpose language, so it is used in many different fields.

Web development: sites and web apps are built with frameworks like Django and Flask, which handle pages, users and databases quickly.

Data science and AI: Numpy, Pandas and scikit-learn are used for data analysis and machine learning models.

Automation and scripting: short scripts do boring repetitive tasks like renaming files, sending emails or scraping web pages automatically.

Education and research: simple English-like syntax makes it the first language taught in many schools and widely used by scientists.

Game development: libraries like Pygame help build simple 2D games.

Desktop applications: tools like Tkinter and PyQt create GUI apps that run on the computer.

Cybersecurity: Python is used to write security tools, scanners and penetration-testing scripts.

So from websites to robots, Python fits almost everywhere, which is why companies everywhere hire Python developers.

3 Why is Python called an interpreted language?

IMPORTANT

An interpreted language is one whose code is run line by line by a program called an interpreter, without a separate compilation step.

A compiled language like C must first be fully translated into machine code by a compiler before it can run.

In Python, the source code is first converted into an intermediate form called bytecode.

Then the Python Virtual Machine (PVM) reads the bytecode and executes it line by line.

You can even type commands one by one in the interactive shell (the >>> prompt) and see each result at once.

This means you can run a program, see an error, fix it and run again immediately, with no

waiting for compilation.

That is why testing and development in Python is fast and beginner friendly.

So Python is called an interpreted language because its code is executed by an interpreter (the PVM) instead of being compiled to machine code first.

4 What are the different ways to run Python?

IMPORTANT

Interactive mode: type python3 in the terminal and run one line at a time; best for quick experiments.

Script mode: write code in a .py file and run it with python3 file.py; the normal way to run full programs.

IDEs and editors: tools like VS Code, PyCharm or IDLE run Python with debugging and code completion.

Jupyter notebooks: code runs in cells with output shown below each cell; popular in data science.

Online interpreters: websites that run Python in the browser with nothing installed.

For exams, remember interactive mode and script mode as the two basic ways of running Python.

5 What is the extension of a Python file?

IMPORTANT

Python source files use the .py extension, for example program.py.

The .py extension tells the operating system and the interpreter that the file holds Python code.

Related extensions: .pyc for compiled bytecode files that Python creates automatically, and .pyw for windowed scripts on Windows.

In answers and file handling questions, always name Python files with the .py extension.

6 **Who developed Python and when was it released?**

Python was developed by Guido van Rossum, a Dutch programmer.

He started the project in the late 1980s and released the first version, Python 0.9.0, in 1991.

Python was named after the British comedy show Monty Python, not the snake.

Van Rossum led the language for many years as its "Benevolent Dictator For Life" (BDFL) until 2018.

Because Python was free and open source, thousands of developers around the world contributed to it.

The release of Python 2.0 (2000) and Python 3.0 (2008) made it a major modern language.

Today Python is one of the most popular languages in the world.

Remember three facts for full marks: creator = Guido van Rossum, year = 1991, name from = the Monty Python show.

TOPIC 2

Structure of a Program, Identifiers and Keywords

2

A Python program is a set of statements executed one after another from top to bottom.

No semicolons or curly braces are needed; indentation (spaces) defines blocks of code.

An identifier is a name for variables, functions or classes; it must start with a letter or underscore.

Keywords are reserved words like if, for, while, def, class and import; they cannot be used as names.

Python is case sensitive: Name and name are two different identifiers.

QUESTION BANK

7 **What is an identifier? State the rules for naming identifiers.**

IMPORTANT

An identifier is a name given to program elements like variables, functions, classes and modules.

Example: in total = 10, the name total is an identifier.

Rule 1: it can contain letters (a-z, A-Z), digits (0-9) and underscores (_).

Rule 2: it must start with a letter or an underscore, never with a digit, so _name and name1 are valid but 1name is not.

Rule 3: it cannot be a keyword like if, for, while or class, because those have fixed meanings in the language.

Rule 4: Python is case sensitive, so Name and name are two different identifiers.

Rule 5: no spaces or special symbols like @, - or $ are allowed in a name.

Good practice: use meaningful names like total_marks instead of single letters like x, so the code is easy to read.

8 What are keywords? Give any five examples.

IMPORTANT

Keywords are reserved words in Python that have a fixed meaning and purpose.

They are part of the language itself, so you cannot use them as variable or function names.

Five examples: if (decision making), else (alternative branch), for (looping), while (condition loop) and def (defining a function).

More examples: return, import, class, True, False and None.

If you try to write name = if, Python gives a syntax error because if is reserved.

There are about 35 keywords in Python 3, and the list is fixed for the language.

You can see all of them by running help("keywords") in the Python shell.

In short: keywords belong to Python, so programmers must pick other names for their own variables.

9 Explain the structure of a Python program.

IMPORTANT

A Python program is a set of statements executed one after another from top to bottom.

A statement is a single instruction, like x = 10.

Related statements are grouped into a block using the same indentation (spaces at the start of the line).

Python needs no semicolons to end lines and no curly braces for blocks, unlike C or Java.

A typical program has: comments, import statements, variable assignments, function definitions and the main logic.

Example code:

```
x = 10
if x > 5:
print("big")
```

Here the print statement is inside the if block because it is indented one level deeper.

If the indentation is wrong, Python gives an IndentationError, so neat spacing is part of the structure.

10 **Demonstrate the significance of indentation in Python programming.**

IMPORTANT

In most languages braces mark a block of code; in Python, indentation (spaces at the line start) does that job.

The interpreter decides which statements belong to an if, for, while or function body purely from their indentation.

Wrong or inconsistent indentation causes an IndentationError and the program will not run at all.

Standard practice is 4 spaces per level; mixing tabs and spaces in one file is an error.

Example:

```
age = 18
if age >= 18:
print("Adult")      # indented: inside the if block
print("Done")           # not indented: outside the if block
```

So indentation in Python is not decoration; it is syntax that defines the structure of the program.

TOPIC 3

3

Operators and Precedence

Arithmetic operators: + - * / // (floor division) % (remainder) ** (power).

Comparison operators: == != > < >= <= give True or False.

Logical operators: and, or, not combine conditions.

Assignment operators: = += -= *= /= assign and update values.

Precedence order: ** first, then * / // %, then + -, then comparisons, then not, and, or last. Use brackets to force order.

QUESTION BANK

11 Distinguish between / and // and between = and ==. (Model Paper, Mar 2024)

REPEATED

/ is normal division and always returns a float, even for whole results: 7 / 2 gives 3.5.

// is floor division: it divides and drops the fractional part, rounding down: 7 // 2 gives 3.

Note the rounding direction: -7 // 2 gives -4, because floor means the next lower whole number.

= is the assignment operator: it stores a value in a variable. Example: x = 5 puts 5 into x.

== is the comparison operator: it checks whether two values are equal and returns True or False.

Example: x == 5 tests the value of x; if x is 5 the result is True.

A common beginner mistake is writing if x = 5 instead of if x == 5; the first one is a syntax error because assignment is not a test.

Remember the trick: single = stores, double == compares.

12 Explain the different types of operators in Python with examples.

IMPORTANT

An operator is a symbol that performs an operation on values called operands.

Arithmetic operators do maths: + - * / // % **. Example: 7 // 2 gives 3, because floor division drops the fractional part.

Comparison operators compare values and give True or False: == != > < >= <=. Example: 5 > 3 gives True.

Logical operators combine conditions: and, or, not. Example: (5 > 3) and (2 < 4) gives True.

Assignment operators store values: = stores; += -= *= /= update and store back. Example: x = 5 then x += 2 makes x equal 7.

Identity operators: is and is not check whether two names point to the same object in memory.

Membership operators: in and not in check whether a value exists inside a sequence. Example: "a" in "cat" gives True.

Bitwise operators (&, |, ^, <<, >>) work on the binary digits of numbers.

These operator families cover almost every operation you will write in a Python program.

13 What is operator precedence? Evaluate: 2 + 3 * 4 ** 2.

IMPORTANT

Operator precedence is the rule that decides which operator is evaluated first in an expression.

Python follows this order: ** (power) first, then * / // %, then + -, then comparison operators, then not, and, or last.

Brackets can force any order you want.

Now evaluate 2 + 3 * 4 ** 2 step by step.

Step 1: power first, 4 ** 2 = 16.

Step 2: multiplication next, 3 * 16 = 48.

Step 3: addition last, 2 + 48 = 50.

So the final answer is 50.

If we wanted addition first we would write (2 + 3) * 4 ** 2, which gives 5 * 16 = 80, showing why precedence matters.

14 What are the purposes of the // and % operators in Python?

IMPORTANT

// is floor division: it divides and drops the fractional part, giving how many whole times one number fits in another.

% is the modulus operator: it gives the remainder left after division.

Example:

```
print(17 // 5)   # 3, because 5 fits 3 full times in 17
print(17 % 5)    # 2, the remainder
```

Common uses: // for splitting items into full groups, % for checking even or odd with n % 2 == 0.

Note the difference from / which gives the exact float result: 17 / 5 = 3.4.

In short: // answers how many full times, % answers what is left over.

15 **Explain the characteristics of membership operators in Python.**

IMPORTANT

Membership operators test whether a value exists inside a sequence like a string, list or tuple.

in returns True when the value is found, for example 'a' in 'apple' is True.

not in returns True when the value is absent, for example 'z' not in 'apple' is True.

They work on strings (substring check), lists, tuples, dictionaries (checks keys) and sets.

Example:

```
fruits = ["apple", "mango"]
print("mango" in fruits)        # True
print("grape" not in fruits)    # True
```

Membership operators are the standard way to check presence before using or removing an item.

16 **Write a Python program to print the result of 5+3*2 and then modify it to get 16.**

IMPORTANT

Python follows operator precedence: * is evaluated before +, like BODMAS in maths.

So 5+3*2 is computed as 5+(3*2) = 5+6 = 11.

To force the addition first and get 16, use brackets: (5+3)*2 = 8*2 = 16.

```
print(5+3*2)      # 11, because * runs before +
print((5+3)*2)    # 16, brackets change the order
```

Brackets always win over the default precedence order.

17 **Explain the truth tables of and, or and not in Python with examples.**

IMPORTANT

and gives True only when both sides are True: True and True is True, every other combination is False.

or gives True when at least one side is True: only False or False is False.

not flips the value: not True is False and not False is True.

Example:

```
print(True and False)   # False
print(True or False)    # True
print(not True)         # False
```

In programs they combine conditions, for example: if age >= 18 and has_id: allow entry.

Remember: and needs both true, or needs one true, not reverses the result.

TOPIC 4

4

Basic Data Types and Type Conversion

int stores whole numbers like 10; float stores decimal numbers like 3.14; complex stores complex numbers like 2 + 3j.

str stores text in quotes like "hello"; bool stores only True or False.

type() shows the data type of a value: type(10) gives int.

Conversion functions change one type to another: int(), float(), str(), bool().

Examples: int("10") gives 10; str(25) gives "25"; float(7) gives 7.0.

QUESTION BANK

18 **Explain the basic data types in Python.**

IMPORTANT

A data type tells Python what kind of value is stored and what operations are allowed on it.

int: whole numbers, positive or negative, e.g. 10, -5.

float: decimal numbers, e.g. 3.14, -0.5.

complex: complex numbers with real and imaginary parts, e.g. 2 + 3j.

str: text written in quotes, e.g. "hello".

bool: logical values, only True or False, used in conditions.

Python finds the type automatically; you do not declare it like in C or Java.

Use type() to check: type(10) gives int and type("hi") gives str.

Knowing the type matters because only matching types can be combined in operations.

19 **What is type conversion? Explain with examples.**

IMPORTANT

Type conversion means changing a value from one data type to another.

Python gives built-in functions for this: int(), float(), str() and bool().

int("10") converts the string "10" to the integer 10.

float("3.5") gives 3.5, and float(7) gives 7.0.

str(100) converts the number 100 to the string "100".

This is needed because input() always returns a string, even when the user types digits.

Example code:

```
age = int(input("Enter age: "))
```

Without int(), age + 1 would fail because you cannot add a number to a string.

So type conversion connects user input with numeric calculations.

20 **What will be the output of int('25') + float('2.5')? Explain.**

IMPORTANT

int("25") converts the string "25" into the integer 25.

float("2.5") converts the string "2.5" into the decimal number 2.5.

Now the expression is 25 + 2.5, with both operands being numbers, so Python adds them.

When an int and a float are mixed, Python promotes the result to float.

25 + 2.5 = 27.5, so the output is 27.5.

Without conversion, "25" + "2.5" would join the two strings into "252.5" instead of adding them.

So conversion functions like int() and float() turn string data into real numbers first.

This example shows conversion turning string data into numbers before maths can happen.

TOPIC 5

Statements, Expressions, Input and Output

5

A statement is one complete instruction, like x = 10.

An expression is a combination of values and operators that produces a result, like 2 + 3 * 4.

print() shows output on the screen; input() reads text typed by the user.

input() always returns a string, so convert it: int(input("Enter age: ")).

Comments start with # and are ignored by Python; they explain the code.

21 Distinguish between a statement and an expression with examples.

IMPORTANT

A statement is a complete instruction that performs an action, like x = 10.

An expression is a combination of values, variables and operators that produces a value, like 2 + 3 * 4.

Every expression evaluates to a single value; for example 2 + 3 gives the value 5.

A statement may or may not produce a value: x = 10 performs the action of storing, but gives no value back.

Example: in y = 2 + 3, the whole line is a statement, and 2 + 3 inside it is an expression.

print("hi") is a statement that displays output; "hello" + "hi" is an expression that builds the value "hellohi".

print(2 + 3) shows 5, because the expression 2 + 3 is evaluated first and its value is printed.

In short: expressions calculate values, statements do things.

22 Explain input() and print() with examples.

IMPORTANT

print() displays output on the screen.

It can show text, numbers and variables together: print("Hello", 2026) prints Hello 2026.

input() reads one line of text typed by the user and returns it.

Example: name = input("Enter name: ") waits for typing and stores it in name.

The text inside input() is called the prompt; it tells the user what to type.

Important: input() always returns a string, even if the user types digits.

So for numbers you must convert the result:

```
age = int(input("Enter age: "))
print("Next year you will be", age + 1)
```

Together, input() takes data in and print() sends results out, which is how a program talks to the user.

23 **Why do we write int(input()) instead of only input()?**

IMPORTANT

input() alone always returns a string, and strings cannot be used in maths directly.

If the user types 5, input() gives the string "5", not the number 5.

Adding strings joins them: "5" + "3" gives "53", not 8.

int(input()) converts the typed text into an integer first, so numeric operations work correctly.

Example: marks = int(input("Enter marks: ")) followed by total = marks + 10 works as real maths.

For decimal input use float(input()) instead of int(input()).

A good rule: use int(input()) for whole numbers, float(input()) for decimals, and plain input() for names and text.

So we wrap input() with int() whenever the program needs to calculate with the entered value.

TOPIC 6

6

Strings

A string is text in single, double or triple quotes; triple quotes allow many lines.

Indexing starts at 0: s[0] is the first character, s[-1] is the last.

Slicing picks a part: s[1:4] takes characters from index 1 up to (not including) 4.

Built-in functions: len(), upper(), lower(), strip(), split(), replace(), find().

Operators: + joins strings, * repeats them, in checks membership.

Formatting with f-strings: f"Hello {name}" puts variable values inside text.

QUESTION BANK

24 **Explain string slicing with examples. (Model Paper, MG2DSCCSC100, Mar 2024)**

SURE MARK

Slicing picks out a part of a string using the form s[start:end].

It takes characters from index start up to, but not including, index end.

Indexing starts at 0, so s[0] is the first character of the string.

Example code:

```
s = 'Python'
print(s[1:4])
```

Indexes 1, 2, 3 hold y, t, h, so the output is yth.

Shortcuts: s[:3] takes from the start and gives Pyt; s[2:] takes to the end and gives thon.

s[:] copies the whole string; s[::-1] reverses it and gives nohtyP.

Negative indexes count from the end: s[-1] is n, and s[-4:-1] gives tho.

Slicing never changes the original string; it returns a new one, because strings are immutable.

25 Explain any five built-in string functions with examples.

IMPORTANT

Strings come with many built-in methods for common text jobs.

len("hello") gives 5, the number of characters in the string.

"hello".upper() gives HELLO; "HELLO".lower() gives hello; "hello".title() gives Hello.

" a b ".strip() removes spaces from both ends, giving "a b".

"a,b,c".split(",") breaks the string at each comma and gives the list ["a", "b", "c"].

"hello".replace("l", "L") gives heLLo, replacing every l with L.

"hello".find("l") gives 2, the index of the first l in the string.

"123".isdigit() gives True while "abc123".isdigit() gives False, testing the content.

These methods return new strings; the original string is never modified.

26 What is string formatting? Explain with an example.

IMPORTANT

String formatting means inserting variable values neatly inside text.

The easiest way is an f-string: put f before the opening quote and write variables inside curly braces.

Example code:

```
name = "Aben"
marks = 95
print(f"{name} scored {marks} marks")
```

Python replaces {name} with Aben and {marks} with 95.

Output: Aben scored 95 marks.

You can even calculate inside the braces: f"{marks + 5}" gives 100.

Older styles are the format() method ("{} scored {}".format(name, marks)) and the % operator, but f-strings are the modern standard.

Formatting keeps output readable without messy + joining and str() calls.

27 **Write the output of: s = 'Python'; print(s[1:4], s * 2).**

IMPORTANT

First, s[1:4] slices the string from index 1 up to index 4 (not included).

Indexes 1, 2, 3 of "Python" are y, t, h, so this gives yth.

Remember: the slice stops before the end index, so the character at index 4 (o) is not included.

Next, s * 2 uses the repetition operator, which repeats the whole string twice.

"Python" repeated twice is PythonPython.

Note that * with a string repeats it, while * with numbers multiplies, so the context decides its meaning.

print() shows both values separated by a single space by default.

So the full output is: yth PythonPython.

28 **How do you access individual characters in a string? Explain with examples.**

IMPORTANT

Characters in a string are accessed by index in square brackets, starting from 0.

s[0] is the first character, s[1] the second, and so on.

Negative indexing counts from the end: s[-1] is the last character.

Example:

```
s = "Python"
print(s[0])    # P
print(s[-1])   # n
```

Going past the last index raises an IndexError.

Indexing with 0 to n-1 and negatives is the basic way to read single characters.

29 **Write a Python program to display the letters of a given string.**

IMPORTANT

A for loop can walk through a string one character at a time.

```
word = input("Enter a string: ")
for ch in word:
print(ch)
```

Each pass of the loop puts the next character into ch and prints it on its own line.

The same can be done with indexes: for i in range(len(word)): print(word[i]).

Looping directly over the string is the simplest letter-by-letter display.

30 **What will be the output of print(3 * 'Python' + '3')? Explain.**

IMPORTANT

* with a string repeats it: 3 * 'Python' makes 'PythonPythonPython'.

+ joins two strings: 'PythonPythonPython' + '3' gives 'PythonPythonPython3'.

So the output is PythonPythonPython3.

Note that '3' is a string here, not a number, so no arithmetic happens.

Rule to remember: string * number repeats, string + string joins.

TOPIC 7

7

Control Flow

if, elif and else run code only when a condition is True.

A for loop repeats over a sequence: for i in range(5) runs 5 times.

A while loop repeats until its condition becomes False.

Loops can be nested: one loop written inside another.

break stops the loop fully; continue skips to the next round; pass does nothing and fills empty blocks.

exit() stops the whole program immediately.

QUESTION BANK

31 **Explain break, continue and pass with examples. (Model Paper, MG2DSCCSC100)**

REPEATED

break ends the loop immediately, even if rounds remain.

continue skips the rest of the current round and jumps to the next round of the loop.

pass does nothing; it is a placeholder where Python needs a statement but you have nothing to write yet.

Example code:

```
for i in range(5):
if i == 3:
break
print(i)
```

This prints 0 1 2 and stops when i becomes 3.

If break were replaced with continue, it would print 0 1 2 4, skipping only 3.

pass is used in empty function or class bodies to avoid a syntax error.

Together, break, continue and pass give full control over how a loop behaves.

32 Explain if-elif-else with an example.

IMPORTANT

if, elif and else let the program choose which code to run based on conditions.

if checks the first condition; if it is True its block runs.

elif (short for else if) checks the next condition only when all previous ones were False.

else runs when every condition above it failed; it needs no condition of its own.

Example code:

```
marks = 75
if marks >= 90:
print("A grade")
elif marks >= 60:
print("B grade")
else:
print("C grade")
```

75 is not >= 90 but is >= 60, so the output is B grade.

You can have many elifs but only one else, and the first True block runs while the rest are skipped.

33 Distinguish between for and while loops with examples.

IMPORTANT

Both loops repeat code, but they are used in different situations.

for is used when the number of repeats is known; it walks over a sequence.

while is used when repeats depend on a condition; it runs until the condition becomes False.

Example code:

```
for i in range(3):
print(i)
```

This prints 0 1 2, because range(3) gives 0, 1, 2.

```
n = 3
while n > 0:
print(n)
n = n - 1
```

This prints 3 2 1, stopping when n reaches 0.

Use for for counting and scanning lists; use while for waiting on changing conditions like user input.

A while loop with a condition that never becomes False runs forever, so the loop variable must be updated inside.

34 What is nested control flow? Give an example.

IMPORTANT

Nested control flow means writing one if statement or loop inside another.

The inner block runs fully for every single round of the outer block.

Example code:

```
for i in range(1, 4):
for j in range(1, 4):
print(i, "x", j, "=", i * j)
```

The outer loop picks i = 1, then the inner loop runs j = 1, 2, 3 completely before i becomes 2.

This prints a small multiplication table: 1x1=1, 1x2=2, 1x3=3, 2x1=2 and so on.

Nesting if inside if works the same way: the inner if is checked only when the outer condition is True.

Deep nesting makes code hard to read, so two levels is usually enough.

Nested loops are essential for tables, patterns and grid-like data.

35 Write a Python program to check whether a given year is a leap year.

IMPORTANT

A year is a leap year if it is divisible by 4, except century years which must be divisible by 400.

```
year = int(input("Enter year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
print("Leap year")
else:
print("Not a leap year")
```

year % 4 == 0 checks divisibility by 4; year % 100 != 0 removes most century years.

year % 400 == 0 keeps century years like 2000 that really are leap years.

The combined condition handles all three rules in one if statement.

36 **Rewrite the following for loop as a while loop with the same output: for i in range(1, 6): print(i * '*').**

IMPORTANT

A for loop with range(1, 6) runs i through 1, 2, 3, 4, 5; a while loop needs a manual start, condition and increment.

```
i = 1
while i < 6:
print(i * "*")
i += 1
```

i = 1 sets the same start; while i < 6 keeps the same end; i += 1 moves to the next value.

The output is identical: *, **, ***, ****, *****.

Every for-range loop can be rewritten as while by managing the counter yourself.

37 **Write a Python program to print the first 10 even numbers using a for loop.**

IMPORTANT

Even numbers are 2, 4, 6 and so on; range(2, 21, 2) steps through them directly.

```
for i in range(2, 21, 2):
print(i)
```

range(2, 21, 2) starts at 2, stops before 21 and steps by 2, giving exactly 10 numbers.

Alternative: loop 1 to 20 and print i only when i % 2 == 0.

A stepped range is the cleanest way to generate even numbers.

38 **Write a Python program that accepts a number and prints whether it is positive, negative or zero.**

IMPORTANT

if-elif-else picks exactly one of the three cases.

```
n = int(input("Enter a number: "))
if n > 0:
print("Positive")
elif n < 0:
print("Negative")
else:
print("Zero")
```

The conditions are checked top to bottom and the first true branch runs.

Three-way tests like this are the textbook use of if-elif-else.

39 List the decision making statements in Python.

IMPORTANT

if: runs a block only when its condition is true.

if-else: picks one of two blocks based on the condition.

if-elif-else: picks one branch out of many ordered conditions.

nested if: an if statement placed inside another if for layered decisions.

Example:

```
marks = 75
if marks >= 90:
print("A grade")
elif marks >= 60:
print("B grade")
else:
print("C grade")
```

These four forms cover every branching need in a program.

40 Explain finite loop and infinite loop with examples.

IMPORTANT

A finite loop runs a fixed, known number of times and then stops by itself.

Example of a finite loop: for i in range(5): print(i) always ends after 5 passes.

An infinite loop has a condition that never becomes false, so it runs forever unless broken.

Example of an infinite loop:

```
while True:
print("runs forever")
break   # without this break it would never stop
```

Infinite loops are sometimes wanted (servers, menus) but usually signal a missing update or a wrong condition.

Finite loops end on their own; infinite loops need a break or a fixed condition to stop.

41 Write a Python program to display a star pyramid pattern.

IMPORTANT

Each row i prints (n-i) spaces then (2*i-1) stars, forming a centered pyramid.

```
n = 5
for i in range(1, n + 1):
print(" " * (n - i) + "*" * (2 * i - 1))
```

" " * (n - i) pushes the stars right so the pyramid is centered.

"*" * (2 * i - 1) grows the star count 1, 3, 5, 7, 9 down the rows.

Spaces for shape plus stars for width is the standard pattern trick.

42 Write a Python program to check whether a number is an Armstrong number.

IMPORTANT

An Armstrong number equals the sum of its digits each raised to the power of the digit count (153 = 1^3+5^3+3^3).

```
n = int(input("Enter number: "))
digits = str(n)
power = len(digits)
total = sum(int(d) ** power for d in digits)
if total == n:
print("Armstrong number")
else:
print("Not an Armstrong number")
```

len() of the digit string gives the power; each digit is raised to it and added.

Compare the computed sum with the original number to decide.

43 Write the syntax of for and while loops and print the factorial of n using both.

IMPORTANT

for syntax: for variable in sequence: with the loop body indented below it.

while syntax: while condition: with the body indented, plus code that moves toward ending.

Factorial with for:

```
n = int(input("Enter n: "))
fact = 1
for i in range(1, n + 1):
fact *= i
```

```
print("Factorial:", fact)
```

Factorial with while:

```
n = int(input("Enter n: "))
fact = 1
i = 1
while i <= n:
fact *= i
i += 1
print("Factorial:", fact)
```

Both versions multiply 1 to n; for manages the counter while while needs a manual i += 1.